

```
class Vehicle:

    def __init__(self, wheels, make, model, color):
        self.__vehType = "Vehicle"
        self.__wheels = wheels    # Private Instance
        self._make = make        # Protected Instance
        self._model = model      # Protectd Instance
        self.color = color       # Public Instance

    def get_wheels(self):
        return self.__wheels

    def get_info(self):
        return f"Wheels : {self.__wheels}, Make : {self._make}, Model : {self._model}, Color : {self.color}"

    def move(self, txt="The"):
        print(f"{txt} {self.__vehType} is moving.")
```

```
class Car(Vehicle):
    def __init__(self, wheels, make, model, color):
        super().__init__(wheels, make, model, color)
        self.__vehType = "Car"

    def move(self, txt="The"):
        print(f"{txt} {self.get_vehType()} is being driven.")

class Motorcycle(Vehicle):
    def __init__(self, wheels, make, model, color):
        super().__init__(wheels, make, model, color)
        self.vehType = "Motorcycle"

    def move(self, txt="The"):
        print(f"{txt} {self.get_vehType()} is being ridden.")
```

```
mycar = Car(4, "Toyota", "Corolla", "Red")
mycar._model
```

```
'Corolla'
```

```
print(f"Is is {mycar._make} {mycar._model}")
```

```
Is is Toyota Corolla
```

```
info = mycar.get_info()
print(info)
```

```
Wheels : 4, Make : Toyota, Model : Corolla, Color : Red
```

```
mybike = Motorcycle(2, "Honda", "CBR", "Blue")
```

✓ Duck Typing

```
objLst = [mycar, mybike]
for obj in objLst:
    info = obj.get_info()
    obj.move("My " + info)
```

```
My Wheels : 4, Make : Toyota, Model : Corolla, Color : Red Vehicle is being driven.
My Wheels : 2, Make : Honda, Model : CBR, Color : Blue Vehicle is being ridden.
```

✓ Name Mangling

```
print(f"It isa a {mycar.get_wheels()} wheeler {mycar._Car__vehType} {mycar._Vehicle__vehType}") # getting vehType without defir

mycar._Vehicle__vehType
```

```

-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_9439/1504078552.py in <cell line: 0>()
----> 1 print(f"It isa a {mycar.get_wheels()} wheeler {mycar._Car__vehType} {mycar._Vehicle__vehType}") # getting vehType
without defining setter and getter of ehicle
      2
      3 mycar._Vehicle__vehType

AttributeError: 'Car' object has no attribute '_Car__vehType'

```

✓ Concreate Class

```

from abc import ABC, abstractmethod
class Vehicle(ABC):

    def __init__(self, wheels, make, model, color):
        self.__vehType = "Vehicle"
        self.__wheels = wheels    # Private Instance
        self._make = make        # Protected Instance
        self._model = model      # Protectd Instance
        self.color = color       # Public Instance

    def set_vehType(self, vehType): # setter are usually "procedures"
        self.__vehType = vehType

    def get_vehType(self):
        return self.__vehType

    def get_wheels(self):
        return self.__wheels
    def get_info(self):
        return f"Wheels : {self.__wheels}, Make : {self._make}, Model : {self._model}, Color : {self.color}"

    @abstractmethod
    def move(self, txt="The"):
        pass

```

```
veh = Vehicle(4, "Toyota", "Corolla", "Red")
```

```

-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_9439/3712859223.py in <cell line: 0>()
----> 1 veh = Vehicle(4, "Toyota", "Corolla", "Red")

TypeError: Can't instantiate abstract class Vehicle without an implementation for abstract method 'move'

```

```

class Car(Vehicle):
    def __init__(self, wheels, make, model, color):
        super().__init__(wheels, make, model, color)
        self.set_vehType("Car")
    def move(self, txt="The"):
        print(f"{txt} {self.get_vehType()} is being driven.")

class Motorcycle(Vehicle):
    def __init__(self, wheels, make, model, color):
        super().__init__(wheels, make, model, color)
        self.set_vehType("Motorcycle")

    def move(self, txt="The"):
        print(f"{txt} {self.get_vehType()} is being ridden.")

```

```
mycar = Car(4, "Toyota", "Corolla", "Red")
mybike = Motorcycle(2, "Honda", "CBR", "Blue")
```

```
mycar.get_info()
```

```
'Wheels : 4, Make : Toyota, Model : Corolla, Color : Red'
```

```
mycar.move("My")
```

My Car is being driven.

```
mybike.move("your")
```

your Motorcycle is being ridden.

```
mybike.get_info()
```

'Wheels : 2, Make : Honda, Model : CBR, Color : Blue'

Start coding or [generate](#) with AI.